不瞎编的私有知识库问答助手

姓名：温定浩 学号：2245014148 日期：2026-10-19

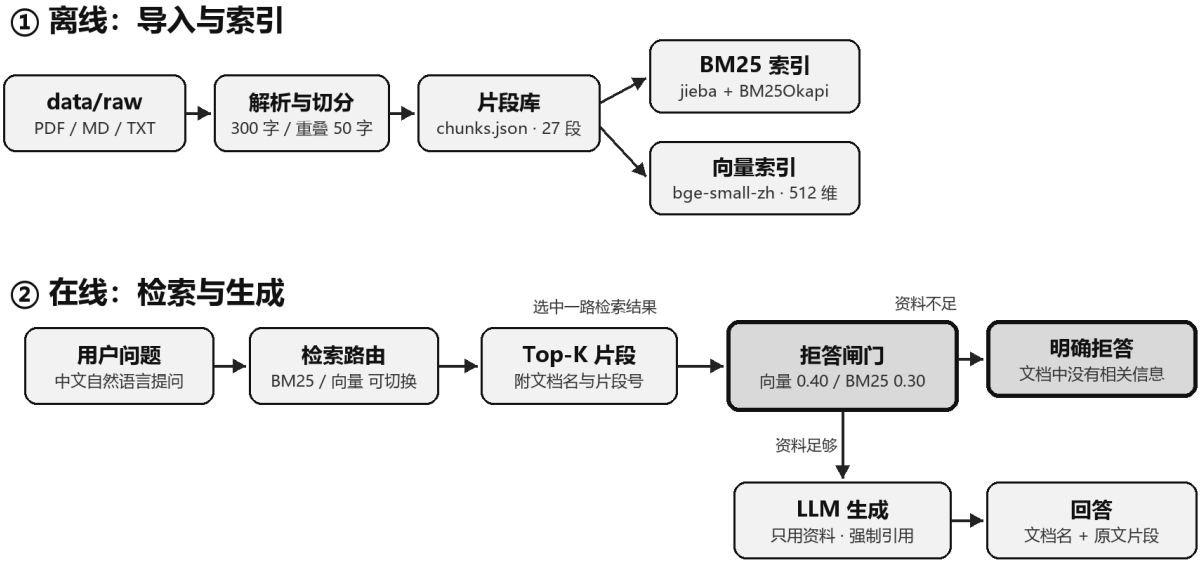
摘要：本实验实现了一个最小可用的个人知识库 RAG 问答助手，支持 PDF、Markdown、TXT 导入，采用 300 字切分，并可在运行时切换 BM25 与 bge-small-zh 向量检索。系统在生成阶段强制标注原文出处，并通过检索阈值与生成约束两层机制实现拒答。实验使用 20 道自建题（10 道文档题、5 道陷阱题、5 道多跳题）对两种检索各运行一遍，从答案正确率、幻觉率、Top-3 检索命中率和首 token 延迟四个维度进行比较，并定位了一个“检索证据齐全却仍答错”的多跳案例。

关键词：检索增强生成；BM25；向量检索；拒答；引用溯源；评测

1 系统架构

系统分为四层。**语料层**把 PDF、Markdown、TXT 解析为统一文本，按段落或句子优先切成 300 字片段，相邻片段重叠 50 字。**索引层**为同一批片段分别建立 BM25 倒排统计和 512 维归一化向量索引，向量由本地 bge-small-zh-v1.5（fastembed，CPU）生成。**检索层**用统一的 search(query, top_k) 接口封装两个检索器，Web 与 CLI 只改一个参数即可切换。**生成层**把 Top-K 片段编号后拼进提示词，要求每个关键结论标注 [n]；程序只接受落在候选范围内的编号，再把编号还原成“文档名 + 片段号 + 原文片段”。

拒答有两道闸门：检索层用向量 Top-1 相似度（< 0.40）或 BM25 查询词覆盖率（< 0.30）挡住明显域外问题；生成层要求模型在资料不足时只输出 INSUFFICIENT_CONTEXT，程序同时识别中英文拒答措辞。第一层不调用模型、延迟低，第二层负责“词面相关但实际答不了”的陷阱。



图注：检索是 1，生成是后面的 0；加粗框为两层拒答中的检索闸门与最终拒答。

图 1 系统架构：从语料导入、双索引检索到带引用的生成与两层拒答

2 实验设置

表 1 实验设置

项目	设置
语料	3 个文档、5 669 字符、2 种格式；最长一篇按“汉字+英文/数字串”计 1 541 字、纯汉字 1 436，口径见 data/README.md
切分	300 字 / 重叠 50 字；依据是日记中 8 题切分实验：200/300/500 字 Top-3 命中分别为 5/8、7/8、6/8
关键词检索	jieba 搜索模式分词 + rank-bm25 BM25Okapi，Top-4
向量检索	BAAI/bge-small-zh-v1.5，512 维，余弦相似度，Top-4
生成	OpenAI 兼容接口（测试用 Pollinations openai-fast），temperature=0，流式输出；提示词禁止使用资料外知识、强制引用
拒答阈值	向量 Top-1 < 0.40；BM25 查询词覆盖率 < 0.30；阈值在这 20 题上校准（in-sample，限制见 3.1）
评测集	20 题：10 文档题、5 陷阱题、5 多跳题；每题标注标准答案、来源文档、原文 evidence
环境	Windows 11，i5-12400 / 16 GB / 无显卡；Python 3.13.9、gradio 6.30.0、fastembed 0.9.0

3 评测结果

3.1 四项指标

表 2 两种检索方案的四项指标

指标	BM25	向量检索
答案正确率	100%	95%
幻觉率（5 道陷阱题）	0%	0%
Top-3 检索命中率	86.7%	93.3%
首次回答平均延迟（首 token）	13.36 s	10.96 s
端到端平均延迟（参考）	14.23 s	12.04 s
引用给出率	100%（15/15）	100%（15/15）
引用文档准确率	100%	100%

BM25 答对 20/20，向量检索答对 19/20；5 道陷阱题全部正确拒答，因此本轮幻觉率都是 0%。值得注意的是，BM25 的 Top-3 命中率反而低 6.7 个百分点，却拿到更高正确率：它在 q11 的 Top-3 漏掉课程片段，但 Top-4 补回并正确比较；向量在 q11 的 Top-3 已含两段证据，却把“三个切分档的命中率不同”误当成“课程建议与实验结论不一致”。这说明检索命中率与答案正确率不是单调关系，生成阶段的比较推理同样关键。

延迟上 BM25 首 token 13.36 s、端到端 14.23 s，向量为 10.96 s / 12.04 s。评测脚本没有把检索与生成分项计时，因此不能把差异直接归因于检索；免费接口排队是主要嫌疑，且均值受个别 30 s 级离群值影响，TTFT 中位数为 11.62 s / 11.27 s。

引用核验：原始结果有 4 处“编号在候选范围内、但不支撑所引句子”的引用（q02 两种模式各 1 处、q15 BM25 1 处、q04 BM25 1 处多余引用）。新增按句 token 重叠校验后，q02 的引用被重映射到正确的五步片段，q04/q15 的多余引用被删除；15/15 道有答案题仍有引用，引用文档准确率 100%。这项指标是程序按句校验 + 人工抽查的结果，不是模型自报。

限制：阈值是在这 20 题上选择的（in-sample）；5 道陷阱题中只有 q18 被检索闸门直接拒答，q16/q17/q19/q20 都由生成层拒答；20 题单轮、1 题差异和 5 题容量都太小，95% 置信区间很宽。因此 100%/95% 与 0% 只代表本轮观测，不作为泛化结论。

3.2 分题型结果

表 3 分题型结果

题型	题数	BM25 正确率	向量正确率	BM25 首 token	向量首 token
文档题	10	100%	100%	15.08 s	9.63 s
多跳题	5	100%	80%	13.45 s	14.10 s
陷阱题	5	100%	100%	9.84 s	10.50 s

检索层面共有 3 处 Top-3 未命中：BM25 的 q11（课程笔记片段 3 排第 4）和 q13（笔记片段 6 排第 4），向量的 q12（日记片段 6 未进 Top-3）。q13、q12 以及 BM25 的 q11 都因为 Top-4 上下文仍包含证据而答对；只有向量的 q11 在证据齐全时仍答错，说明“检索到了”与“会用证据”是两件事。

3.3 一个成功案例

q14 问“作者为什么强调每个项目建独立虚拟环境？上学期遇到什么版本冲突？”。BM25 Top-3 同时命中 Python 环境配置备忘.txt 的片段 6 和 5（分数 17.10、16.41），回答准确写出“课程 A 要 numpy 1.x、课程 B 要 numpy 2.x，环境崩溃只能重装”，并分别标注 [2]、[1]。这个案例说明：当两段证据都在 Top-3、且问题里的“为什么/什么”与原文措辞接近时，抽取式生成很稳定。

4 两问分析

4.1 两种检索各自在什么类型的问题上表现更好，为什么？

BM25 在**专有名词、数字、命令和原文措辞明确**的文档题上更稳。q04 的 k1、q07 的“1.8 秒 / 4 秒”、q10 的 Set-ExecutionPolicy 都是低频精确串，BM25 的 IDF 会把它们放大，Top-1 分数分别达到 6.29、18.79、37.58，且第一位就是正确片段。它的问题在于词面依赖和分片边界：q11 问“课程建议多长”，课程笔记片段 3 同时包含“200~400 字”和 bge 维度，词被其他主题稀释，只排到第 4；q13 的“向量检索弱点”也落在第 4 个片段。也就是说，BM25 的命中率对“一个片段里混了几个主题”很敏感。

向量检索在**语义改写、概念性提问和跨文档多跳**上更好：q11 的 Top-3 同时召回日记片段 2（0.829）和课程片段 3（0.713），q13 同时召回笔记片段 6、日记片段 3 和笔记片段 5，Top-3 命中率因此比 BM25 高 6.7 个百分点。但高命中率没有转化为更高正确率：q11 的两段证据都在 Top-3，模型却做了错误的“三者比较”；q12 中“第一次加载 6 分钟”所在的日记片段 6 甚至没进 Top-3，只是靠 Top-4 才答对。向量检索还会把“主题相关”误当成“答案在此”，q11 就是典型：相似度很高，但模型没有把“课程建议的范围”与“实验选的点值”对齐。总体看，**精确事实优先 BM25，口语化或跨片段优先向量**；但检索分数只能保证“材料相关”，不能保证“模型会正确使用材料”，两者都需要来源感知和比较推理的后处理。

4.2 系统最失败的一个案例是什么，根本原因是什么？

最失败的案例是 q11：“课程建议中文文档按多长切分？作者实测 200、300、500 字的 Top-3 命中率分别是多少？两者一致吗？”最终一轮里，BM25 的 Top-3 是日记片段 2（32.93）、片段 1（25.81）、片段 3（11.60），课程笔记片段 3 只排第 4；它靠第 4 个候选补出“200~400 字”，答对 5/8、7/8、6/8，并正确判断“课程范围包含 300，故相符”。向量检索的 Top-3 反而更完整：日记片段 2（0.829）、课程片段 3（0.713）、课程片段 8（0.682）都在，但它只复述了“200~400 字”和三个命中率，最后写“三者并不一致，300 字切分表现最佳”，没有回答“课程建议与实验结论是否一致”。标准答案是：课程建议 200~400 字，作者实验 300 字最好，300 落在课程区间内，二者方向一致。

根因有三层。第一层是**问题指代**：题里的“两者”指“课程建议”和“实验结论”，模型却把它解析成“200/300/500 三个切分档”，于是比较了三个本就不同的命中率，答非所问。第二层是**生成协议**：提示词只要求“抽取 + 引用”，没有要求“先重述比较对象，再逐项列证据，最后判断包含或矛盾”；“300 属于 200~400”需要一步集合包含推理，模型没有做。第三层是**检索与切分**：课程片段 3 同时塞进切分建议、bge 维度、chroma 和 Top-K，主题被稀释，BM25 只排到第 4；向量虽然召回，但高相似度只说明“主题相关”，不能保证模型会把两段证据用于同一个判断。两次同配置运行也说明其脆弱性：前一轮 BM25 和向量都答成“不一致”，这一轮 BM25 答对、向量仍错，正确性依赖候选顺序和模型发挥。验证方法是把 q11 拆成“课程建议是什么”“实验最优是多少”两个子查询，分别核对片段 3 和片段 2，确认两条事实都存在，因此错误在比较推理而不是语料缺失。修正方向是查询分解与比较模板（先写比较对象，再做区间包含判断），以及按标题或小节切分，避免一个片段混多个主题；不应通过改题或放宽阈值来掩盖。

5 AI 错误清单

表 4 AI 生成代码的主要错误与修正

#	AI 给了什么	错在哪	怎么发现	怎么修正
1	gr.Chatbot(type="messages", show_copy_button=True)	Gradio 5 旧参数，本机 Gradio 6.30 已移除	构造界面报 TypeError: unexpected keyword argument 'type'	查 inspect.signature，改成 buttons=["copy"]
2	断言 query_coverage("莫干山 雪山") == 0.5	忽略 jieba 搜索模式会把“莫干山”再切成子词，实际 0.667	pytest 断言失败，打印 0.666…	改成“部分命中，0 < coverage < 1”，补完全无关为 0 的断言
3	初版向量阈值拍脑袋设 0.35，并声称相关 >0.52、无关 <0.30	阈值没有实测；校准后无关问题 0.509，相关问题下限 0.441，两组重叠	8 个相关 + 8 个无关问题逐条打印 Top-1 分数	阈值改 0.40，只做粗筛；语义陷阱交给生成阶段
4	引用正则只写 \[(\d+)\]	模型实际输出中文括号 【1】，citations 为空	pilot 回答有 【1】，CSV 的 citations 列却为空	正则同时匹配 [] 【】 〔〕，补单测
5	直接用 fastembed 默认临时缓存目录下载模型	沙箱或临时目录权限导致下载反复失败，索引建不起来	ingest 报 PermissionError 或 Could not load model	支持 EMBEDDING_MODEL_PATH 离线模型目录，缓存改到工作区 .cache/
6	流式响应直接取 chunk.choices[0].delta.content	兼容接口会先返回 choices=[] 的 usage/心跳块，触发 IndexError	流式冒烟测试抛 IndexError: list index out of range	跳过空 choices，只拼接非空 delta，并用首个内容块记录首 token

6 总结

怎么利用 AI 重构代码。 我先让 AI 生成最小骨架（解析、切分、BM25、向量、CLI、评测脚本），再用测试和真实运行逐段否决：Gradio 6 的参数、jieba 的分词行为、引用括号、临时目录权限，都是跑起来才暴露的。AI 适合写“可验证的局部”，不适合替我做“阈值是否可信、题是否真的有答案、引用是否指向正确片段”这类判断。每次修改后我都用 pytest、--retrieval-only 或单题 CLI 复现，确认不是“看起来对”。

评测陷阱。 第一，文档级命中会高估检索：q13 的 Top-3 没有向量弱点片段，但同文档另一片段在 Top-4，答案仍然对，若只按文档名判分会看不出缺口。第二，Top-3 与 Top-K 不是一回事：本系统实际给模型 Top-4，所以“Top-3 命中率”低不代表答错，报告必须同时看上下文和答案。第三，陷阱题要先确认语料真的没有答案；“Chroma 版本号”这种题看似可答，实际只提到 chroma、没有版本。第四，q11 暴露了“一致吗”这种比较题容易被模型用字面数字糊弄，出题时应把判定规则写进标准答案。第五，免费接口有 30 秒限流，评测必须限速并记录重试，否则 402 会被误记成模型错误。

AI 时代什么能力最重要。 不是把代码写出来，而是定义可验证的目标、设计能暴露问题的评测，并对“自信的输出”保持怀疑。代码可以生成，检索片段不会说谎；真正稀缺的是判断哪条证据算数、哪个指标被高估、哪个 failure 是检索、切分还是生成的锅。这次实验最大的收获，是把“不瞎编”从一句提示词，变成了阈值、引用校验、20 题评测和一份能复现的失败记录。